

الوراثة

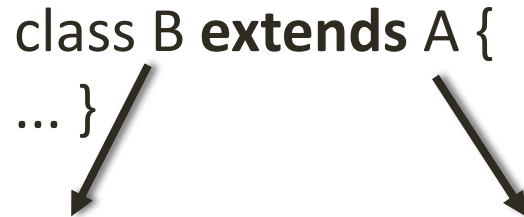
L'héritage

تمهيد

- من أساسيات البرمجة الكائنية
- تمكن من اعادة استعمال البرامج
- تعرف صنف (الصنف المشتق) ابتداءا من صنف موجود (الصنف الأصلي)
- الصنف المشتق يرث وظائف الصنف الأصلي من خصائص وطرق
- امكانية تغيير الوظائف الأصلية كليا أو جزئيا واطافة وظائف أخرى

الصفة

```
class B extends A {  
... }
```



الصفء الوارء
الصفء المءءء
Classe fille
Child class
Sub class

الصفء الموروء
الصفء الأءلى
Classe mère
Parent class
Super class

مثال

```
class Etudiant extends Personne {
```

```
    private String mat;
```

```
    public String getMat(){  
        return mat;  
    }
```

```
    public void setMat(String n){  
        mat=n;  
    }
```

```
}
```

← خاصية جديدة

← طرق جديدة

التطبيق

```
Etudiant E1=new Etudiant();  
E1.afficher();
```

الدالة البانية للصنف المشتق

Personne طريقة في الصنف

النتيجة

Nom : null

Prénom : null

Age : 0

- لا تظهر الخاصية mat
- ادخال دالة جديدة

مثال

```
class Personne
{
    private String nom ;
    private String prenom ;
    private int age ;
```

في هذا المثال لا يمكن للأصناف المشتقة من
Personne الوصول مباشرة لخاصيات

```
public void afficher ()
{
    System.out.println(" Nom : "+ nom)
    System.out.println(" Prenom : "+ prenom);
    System.out.println (" Age : "+age);
}
}
```

```
Class Etudiant extends Personne {
    private String mat;
    public void afficherEtud()
    {
        System.out.println(" Matricule : "+ nom)
        afficher();
    }
}
```

استعمال دالة معرفة في الدالة الأصلية

```
class Personne
```

```
{  
    protected String nom ;  
    protected String prenom ;  
    protected int age ;  
}
```

```
public void afficher ()
```

```
{  
    System.out.println(" Nom : "+ nom)  
    System.out.println(" Prenom : "+ prenom);  
    System.out.println (" Age : "+age);  
}  
}
```

```
Class Etudiant extends Personne {
```

```
    private String mat;  
    public void afficherEtud()  
    {  
        System.out.println(" Matricule : "+ mat)  
        System.out.println(" Nom : "+ nom)  
        System.out.println(" Prenom : "+ prenom);  
        System.out.println (" Age : "+age);  
    }  
}
```

الكلمة المفتاحية protected

protected تمكن الأصناف المشتقة من استعمال خاصيات الأصناف الأصلية

اعادة صياغة الطرق

Redéfinition d'une méthode overriding

```
Class Etudiant extends Personne {  
    private String mat;
```

نلاحظ من المثال السابق أننا نحتاج في كل اشتقاق لاضافة طريقة لعرض المعطيات. يوجد في POO امكانية اعادة صياغة طريقة ما في كل اشتقاق.

```
public void afficher ()  
{  
    System.out.println (" Matricule : "+mat);  
    System.out.println(" Nom : "+ n)  
    System.out.println(" Prénom : "+ p);  
    System.out.println (" Age : "+a);  
}
```

نفس تعريف الدالة الأصلية
(نفس الإمضاء)

```
... }
```

```
public void afficher ()  
{  
    System.out.println (" Matricule : "+mat);  
    super.afficher();  
}
```

الكلمة المفتاحية **super** : **الصف**
الأصلي

التطبيق

```
Etudiant E1=new Etudiant();  
E1.afficher();
```

النتيجة

Matricule : null
Nom : null
Prénom : null
Age : 0

في الكائنات الموروثة :

- كيف تبني الدوال الموروثة ؟
- كيف تستعمل الدوال المعاد صياغتها ؟

الدوال البانية

الحالة العامة

في حالة وجود دوال بانية في الصنفين. الصنف المشتق يبدأ بتطبيق الدالة البانية للصنف الأصلي (الدالة البانية للصنف المشتق تأخذ على عاتقها بناء الصنف الأصلي)

```
class A {  
    ...  
    A(x,y){  
        ...  
    }  
    ...  
}
```

```
class B extends A {  
    ...  
    B(x,y,z){  
        super(x,y);  
        ...  
    }  
    ...  
}
```

الدالة البانية لـ A

مثال

class Personne

```
{ ...  
    Public Personne( String n, String p, int a)  
    {  
        nom =n ;  
        prenom =p ;  
        age =a;  
    }  
...  
}
```

```
Etudiant E1=new Etudiant("KACI","Ali",23, "A010");  
E1.afficher();
```

Matricule : A010
Nom : KACI
Prénom : Ali
Age : 23

class Etudiant extends Personne {

private String mat;

public Etudiant(String n, String p, int a, String m)

{

super(n,p,a);

mat=m;

}

... }

← الدالة البانية للصنف الأصلي

حالات خاصة

حالة عدم وجود دوال بانية

```
class Personne
{
    protected String nom ;
    protected String prenom ;
    protected int age ;

    public void afficher ()
    {
        System.out.println(" Nom : "+ nom)
        System.out.println(" Prénom : "+ prenom);
        System.out.println (" Age : "+age);
    }
}
```

```
Etudiant E1=new Etudiant();
E1.afficher();
```

```
Class Etudiant extends Personne {
    private String mat;

    public void afficher ()
    {
        super.afficher()
        System.out.println (" Mat: "+mat);
    }

    public String getMat(){
        return mat;
    }
    public void setMat(String n){
        mat=n;
    } }
}
```

الدالة البانية التلقائية ل Etudiant تستدعي
الدالة البانية التلقائية ل Personne



أمثلة خاصة

حالة عدم وجود دالة بانية في الدالة المشتقة

```
class Personne
{
    private String nom ;
    private String prenom ;
    private int age ;
```

```
Class Etudiant extends Personne {
    ...
}
```

```
Personne () { ...
}
```

يجب تعريفها وإلا سيكون خطأ في التصريف

```
Personne (String n, String p, int a){
    ...
}
```

```
Etudiant E1=new Etudiant();
E1.afficher();
```

```
public void afficher ()
{
    System.out.println(" Nom : "+ nom)
    System.out.println(" Prénom : "+ prenom);
    System.out.println (" Age : "+age);
}
}
```

أمثلة خاصة

حالة عدم وجود دالة بانية في الدالة الأصلية

```
class Personne
{
    private String nom ;
    private String prenom ;
    private int age ;

    ...

    public void afficher ()
    {
        System.out.println(" Nom : "+ nom)
        System.out.println(" Prénom : "+ prenom);
        System.out.println (" Age : "+age);
    }
}
```

Class **Etudiant** extends **Personne** {

```
Etudiant(){
    super();
}
```

```
...
public void afficher ()
{
    super.afficher()
    System.out.println (" Mat: "+mat);
}
}
```

```
Etudiant E1=new Etudiant();
E1.afficher();
```

الدالة البانية ل Etudiant تستدعي الدالة
البانية التلقائية ل Personne

الاشتقاق المتتالي

class Personne

```
{
    private String nom ;
    private String prenom ;
    private int age ;

    public void afficher ()
    {
        System.out.println(" Nom : "+ nom)
        System.out.println(" Prénom : "+ prenom);
        System.out.println (" Age : "+age);
    }
}
```

Class **Etudiant** extends **Personne** {
private String mat;
...
}

class **EtudEns** extends **Etudiant** {
private String refContratEdu ;

public void afficher()
{
 System.out.println(" Mat : "+ **super.getMat()**);
 super.afficher();
 System.out.println(" Contrat : "+ refContratEdu);
} ...

اجباري

غير اجباري

```
EtudEns E1=new EtudEns();  
E1.afficher();
```

Mat : null
Nom : null
Prénom : null
Age : 0
Contrat: null

إعادة صياغة الطرق

يمكن استعمال طريقة مشتقة عن صنف ما باشتقاق مباشر أو غير مباشر (متتالي)

عند إعادة صياغتها في صنف ما تصبح الطريقة المغيرة هي المستعملة ابتداء من هذا الصنف و في الأصناف المشتقة منه حتى تعاد صياغتها مرة أخرى

```
class A{...  
    private void f()  
    ...  
}
```

```
class B extends A{...}
```

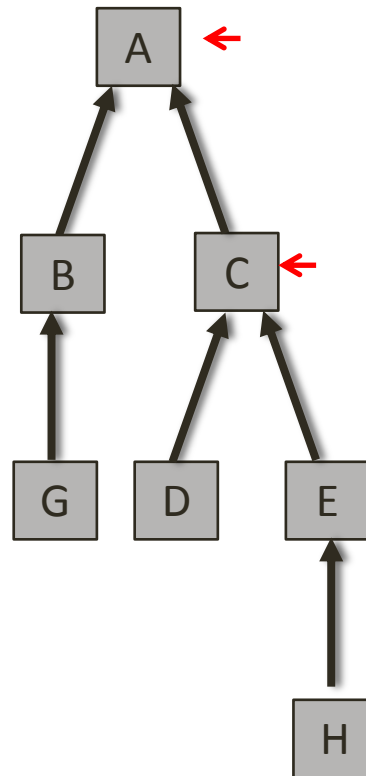
```
class C extends A{...  
    private void f()  
    ...}
```

```
class D extends C{...}
```

```
class E extends C{...}
```

```
class G extends B{...}
```

```
class H extends E{...}
```



```
B b=new B();  
b.f();          // f --> A
```

```
C c=new C();  
c.f();          // f --> C
```

```
D d=new D();  
d.f();          // f --> C
```

```
G g=new G();  
g.f();          // f --> A
```

```
H h=new H();  
h.f();          // f --> C
```

التعريف المتعدد

```
class Personne
```

```
{
```

```
    private String nom ;
```

```
    private String prenom ;
```

```
    private int age ;
```

```
    ...
```

```
    public void afficher ()
```

```
{
```

```
    System.out.println(" Nom : "+ nom)
```

```
    System.out.println(" Prénom : "+ prenom);
```

```
    System.out.println (" Age : "+age);
```

```
}
```

```
}
```

```
Class Etudiant extends Personne {
```

```
    private String mat;
```

```
    public void afficher (String n)
```

```
{
```

```
        super.afficher();
```

```
        System.out.println (" Mat: "+n);
```

```
}
```

```
    public String getMat(){
```

```
        return mat;
```

```
}
```

```
    public void setMat(String n){
```

```
        mat=n;
```

```
    } }
```

من الصنف Personne

```
Etudiant E1=new Etudiant();
```

```
E1.afficher();
```

```
E1.afficher(E1.getMat());
```

من الصنف Etudiant

إعادة التعريف (الصيغة) و تعدد التعاريف

يمكن جمع العمليتين أثناء الاشتقاق ولكن يجدر الحذر من الوقوع في تعقيدات وأخطاء عند استعمال الطرق.

إضافة خاصيات

```
Class Etudiant extends Personne {  
    protected String mat;  
    ...  
}
```

يمكن إضافة خاصيات جديدة عند الاشتقاق

```
Class EtudEns extends Etudiant {  
    public int mat;  
    private String refContratEdu;  
    ...}
```

يمكن تسمية خاصية بخاصية موجودة من الأصناف الموروثة في هذه الحالة تستعمل الخاصيتان والأصلية يضاف لها `super`

```
EtudEns E1=new EtudEns();  
int m=E1.getMat();  
String d=E1.getDiplome();  
String m1=E1.super.getMat();
```

الكلمة المفتاحية final

لإيقاف إمكانية اشتقاق صنف ما نستعمل الكلمة المفتاحية final

الصنف final لا يقبل اشتقاق

```
public final class EtudEns extends Etudiant {...}
```

الطريقة final لا تقبل إعادة التعريف

```
public final void afficher();
```

الخاصية final تحدد لها قيمة وتكون ثابتة

```
public final double PI = 3.14159;
```